```python
import numpy as np

# --- PARAMETER BARU UNTUK PROBABILITAS ---
# Decay constant (k): Semakin tinggi, semakin buruk akurasi jarak jauh.
K_DECAY = 0.1
# Bobot untuk Probabilitas (gamma): Seberapa kuat probabilitas mempengaruhi pilihan rute.
# Jika gamma tinggi, semut akan memprioritaskan target yang 'mudah' dijangkau.
GAMMA = 2

# --- PARAMETER LAMA ---
NUM_CITIES = 5
NUM_ANTS = 10
MAX_ITERATIONS = 100
RHO = 0.5
ALPHA = 1
BETA = 5
INITIAL_PHEROMONE = 1.0
Q = 100

distances = np.array([
    [0, 10, 15, 20, 12],
    [10, 0, 9, 17, 18],
    [15, 9, 0, 8, 11],
    [20, 17, 8, 0, 14],
    [12, 18, 11, 14, 0]
])

# --- 1. HITUNG PROBABILITAS KEBERHASILAN (P) ---
# P = exp(-k * d)
probability_matrix = np.exp(-K_DECAY * distances)
np.fill_diagonal(probability_matrix, 0) # Probabilitas ke diri sendiri = 0

# --- 2. HITUNG HEURISTIK OPTIMAL (ETA PRIMA) ---
# Heuristik = (1/Jarak)^BETA * (Probabilitas)^GAMMA
inverse_distance = 1.0 / (distances + 1e-6)
```

```
np.fill_diagonal(inverse_distance, 0)

# Visibilitas/Heuristik (eta_prime)
eta_prime = (inverse_distance*BETA) * (probability_matrix*GAMMA)

# --- 3. ALGORITMA UTAMA (Dimulai dengan inisialisasi ulang) ---
pheromones = np.full((NUM_CITIES, NUM_CITIES), INITIAL_PHEROMONE)
best_route = None
best_cost = np.inf
best_prob_success = 0.0

def calculate_route_prob(route):
    """Menghitung probabilitas kumulatif sukses dari sebuah
rute."""
    # Probabilitas kumulatif untuk serangkaian tembakan adalah P1 *
P2 * P3 * ...
    prob = 1.0
    for i in range(len(route)):
        city_a = route[i]
        city_b = route[(i + 1) % NUM_CITIES]
        prob *= probability_matrix[city_a][city_b]
    return prob

for iteration in range(MAX_ITERATIONS):
    all_ant_routes = []

    # Gerakan Semut (Sama, tetapi menggunakan eta_prime)
    for ant in range(NUM_ANTS):
        start_city = np.random.randint(NUM_CITIES)
        route = [start_city]
        unvisited = list(range(NUM_CITIES))
        unvisited.remove(start_city)
        current_city = start_city

        while unvisited:
            probabilities = []

            for next_city in unvisited:
```

```
                # Rumus Probabilitas: (Feromon^Alpha) * (Heuristik
Baru^1)
                # Catatan: Bobot Beta sudah dimasukkan ke dalam
eta_prime
                numerator =
(pheromones[current_city][next_city]**ALPHA) * \
                           (eta_prime[current_city][next_city])
                probabilities.append(numerator)

            # Normalisasi dan Pemilihan
            probabilities = np.array(probabilities)
            total_sum = np.sum(probabilities)
            if total_sum == 0:
                next_city_index = np.random.randint(len(unvisited))
            else:
                probabilities /= total_sum
                next_city_index = np.random.choice(len(unvisited),
p=probabilities)

            next_city = unvisited[next_city_index]
            route.append(next_city)
            unvisited.remove(next_city)
            current_city = next_city

        all_ant_routes.append(route)

        # Cek dan update rute terbaik global
        current_cost = calculate_route_cost(route)
        current_prob = calculate_route_prob(route)

        # Kriteria Baru: Prioritaskan probabilitas tinggi.
        # Anda dapat menggunakan kriteria campuran: misalnya,
min_cost * max_prob
        if current_prob > best_prob_success:
             best_cost = current_cost
             best_prob_success = current_prob
             best_route = route

    # Update Feromon (sama seperti sebelumnya)
```

```
    pheromones *= (1.0 - RHO)
    for route in all_ant_routes:
        cost = calculate_route_cost(route)
        # Feromon tetap didasarkan pada biaya/jarak yang rendah
        pheromone_deposit = Q / cost

        for i in range(len(route)):
            city_a = route[i]
            city_b = route[(i + 1) % NUM_CITIES]
            pheromones[city_a][city_b] += pheromone_deposit
            pheromones[city_b][city_a] += pheromone_deposit

# --- HASIL MODIFIKASI ---
print(f"## ⏳ Hasil Optimalisasi Kapal Selam (ACO Modifikasi) ##")
print(f"Konstanta Akurasi (k): {K_DECAY}, Bobot Probabilitas
(gamma): {GAMMA}")
print("---")
print(f"*Matriks Probabilitas Keberhasilan (berdasarkan
jarak):*\n{probability_matrix}\n")
print(f"*Rute Terbaik Ditemukan:* {best_route}")
print(f"*Total Probabilitas Kumulatif Sukses:*
{best_prob_success:.4f}")
print(f"*Total Biaya Rute (Jarak):* {best_cost}")
```

Hasil Run

```
--- Hasil Komputasi Firing Solution (Y = mX + c) ---
Koefisien (m): -5.00
Intercept (c): 125.00

--- Prediksi Penjualan ---
Prediksi Periode ke-11: 70 unit
Prediksi Periode ke-12: 65 unit

--- Titik Nol Profit (BEP = 65 unit) ---
Toko diprediksi masuk zona kerugian di awal Periode Taktis ke-: 12.0
```